Proyecto 01
Chat Multithread

I. Objetivo

Desarrollar una aplicación de chat en C para reforzar los conocimientos sobre procesos, *threads*, concurrencia y comunicación entre procesos.

II. Recomendaciones

Para desarrollar el proyecto se recomienda leer https://linux.die.net/man/7/ip y sus enlaces relacionados, así como investigar el uso de SOCKETS.

III. Condiciones y fechas de entrega:

Grupos: 3 integrantes máximo. Cada grupo producirá un sistema de chat completo (servidor y cliente).
Presentación: el proyecto será ejecutado en equipos conectados a una infraestructura en clase. El día de la entrega cada integrante del grupo debe llevar un cable de red con conectores RJ45 (si su computadora requiere un adaptador, asegúrese de llevarlo).

PARTE 01

Fecha de Entrega: 27 de marzo, 2026. 23:59 Hrs.
Material por entregar: definición del protocolo de comunicación a utilizar entre clientes y servidor.

PARTE 02

Fecha de Entrega: 15 de abril, 2026. Durante los períodos de clase.
Material por entregar: en Canvas antes de la fecha y hora de la presentación. No se otorgará calificación a grupos que no suban su trabajo a Canvas.

- Todos los archivos de código escritos y desarrollados por el grupo, debidamente comentados e identificados.
- Documento en formato PDF con investigación realizada, definiciones de protocolo y documentación que explique el proceso de desarrollo de la solución al proyecto.

IV. Instrucciones

PARTE 01

Establecer el protocolo de comunicación entre clientes y servidores, que utilizará el grupo para garantizar la conectividad e interacción exitosa.

Definir un protocolo simple, funcional y consistente. No se evaluará complejidad sino claridad y correcta implementación. Esto implica establecer los pasos con los que se inicia y termina una conexión, así como el formato (componentes y orden) de cada comunicación entre el cliente y servidor.

PARTE 02

La aplicación debe estar separada dos partes:

1. **Servidor**: mantiene una lista de todos los clientes/usuarios conectados al sistema. Sólo puede existir un servidor durante una ejecución del sistema de chat. El servidor se podrá ejecutar en cualquier máquina Linux que provea la infraestructura necesaria y los clientes se conectarán al servidor mediante la dirección IP de la máquina donde se ejecute el servidor.

2. **Cliente**: se conecta y se registra con el servidor. Un sistema de chat permite la existencia de uno o más clientes durante su ejecución. Posee una pantalla mediante la que se despliegan los mensajes que el usuario ha recibido del servidor, enviados por un usuario remoto; y en la que se permite el ingreso de texto para envío de mensajes a otros usuarios. Cada cliente debe presentar su propia y única pantalla de chat.

El servidor se ejecuta como un proceso independiente mediante el siguiente comando:

```
<nombredelservidor> <puertodelservidor>
```

Donde:
`<nombredelservidor>` es el nombre del programa y
`<puertodelservidor>` es el número de puerto (de la máquina donde se ejecuta el servidor) en el que el servidor está pendiente de conexiones y mensajes de sus clientes.

El servidor debe atender las conexiones de sus clientes con *multithreading*, usando el modelo propuesto en el libro/presentación de ese tema. El servidor no permitirá que existan dos usuarios conectados con el mismo nombre.

El servidor debe proteger mediante mecanismos de sincronización las estructuras de datos compartidas entre threads, especialmente el listado de usuarios conectados y la información asociada a cada sesión. El grupo deberá justificar qué recursos compartidos protegió y por qué.

Además, el documento del proyecto deberá describir al menos dos posibles condiciones de carrera en el servidor y cómo fueron evitadas.

El servidor deberá proveer los siguientes servicios:

- **Registro de usuarios:** el cliente envía al servidor su nombre de usuario. El servidor registra el nombre de usuario junto con la dirección IP de origen en la lista de usuarios conectados *si ni el nombre ni la dirección existen ya en su registro*. De lo contrario, el servidor envía una respuesta que el cliente usa para desplegar un mensaje de error. Al aceptar la conexión del cliente se debe crear su sesión en el servidor y manejarla de forma concurrente a las demás sesiones y a la recepción de nuevas conexiones (*ergo*, *multithreading*).

- **Liberación de usuarios:** cuando un usuario cierra su cliente de chat o elige salir, el procedimiento debe conllevar aviso al servidor (o detección por parte de éste) para el cierre controlado de la sesión asociada. Esto es: que el servidor remueva la información del cliente del listado de usuarios conectados y, consecuentemente, que niegue intentos futuros de comunicación con ese usuario hasta que se registre un nuevo usuario con ese nombre, sin errores.
- **Listado de usuarios conectados**: el cliente podrá solicitar al servidor, en cualquier momento, el listado de usuarios conectados. El servidor responderá con el listado de nombres de usuarios conectados.
- **Obtención de información de usuario:** durante su ejecución, un cliente puede solicitar información de un usuario conectado en específico. La solicitud indica al servidor el nombre de usuario deseado y el servidor responde con la información (IP) asociada con ese nombre, si dicho nombre está conectado al servidor.
- **Cambio de *status*:** el cliente puede presentar cualquiera de los *status* siguientes: ACTIVO, OCUPADO e INACTIVO. ACTIVO es el *status* por defecto, pero un cliente, durante su ejecución, puede solicitar al servidor que cambie su *status*. El cliente siempre reflejará el *status* que el servidor le tenga asignado. El *status* INACTIVO puede ser asignado por el servidor a un cliente que ha pasado un período de inactividad predeterminado (establecido por los desarrolladores). Procuren que el tiempo sea corto o modificable para facilitar la evaluación.
- ***Broadcasting* y mensajes directos:** los usuarios podrán comunicarse mediante el "chat general", donde el servidor envía los mensajes a todos los usuarios conectados, o enviarse mensajes directos, de un usuario específico a otro.

El cliente se ejecutará en mediante el siguiente comando:

```
<nombredelcliente> <nombredeusuario> <IPdelservidor> <puertodelservidor>
```

Donde:
`<nombredelcliente>` es el nombre del programa
`<IPdelservidor>` y `<puertodelservidor>` serán a donde debe llegar la solicitud de conexión del cliente según la configuración del servidor.

El cliente podrá implementar la interfaz que el desarrollador desee, pero deberá proveer al usuario las facilidades para:

1. Chatear con todos los usuarios (broadcasting).
2. Enviar y recibir mensajes directos, privados, aparte del chat general.
3. Cambiar de status.
4. Listar los usuarios conectados al sistema de chat.
5. Desplegar información de un usuario en particular.
6. Ayuda.
7. Salir.

Para chatear con usuarios el formato debe ser similar a lo siguiente: `<usuario> <mensaje>`
Donde
`<usuario>` es el destinatario a quien se entregará el `<mensaje>`.

Cambiar de *status* permitirá al cliente elegir entre ACTIVO, OCUPADO e INACTIVO, y la elección enviará una solicitud de actualización de información al servidor. El cliente deberá refrescar su *status* una vez que el servidor haya realizado el cambio. El listado de usuarios se desplegará como especificado anteriormente.

Componente	Criterio	%
Cliente	Chat general (envío y recepción correcta de mensajes)	5
	Envío de mensajes privados desde el cliente (formato y uso correcto)	5
	Uso de multithreading en el cliente (recepción concurrente mientras el usuario escribe)	5
	Cambio de status (ACTIVO, OCUPADO, INACTIVO)	5
	Listado de usuarios e información de usuario	5
	Interfaz funcional (flujo claro, sin bloqueos, usable)	5
Servidor	Manejo de múltiples clientes con multithreading	10
	Registro de usuarios (validación de duplicados)	5
	Liberación correcta de usuarios (desconexión limpia)	5
	Broadcasting funcional	5
	Enrutamiento correcto de mensajes directos entre usuarios	5
	Manejo de estados (ACTIVO, OCUPADO, INACTIVO)	5
	Respuesta a consultas (listado e info de usuarios)	5
	Uso correcto de sincronización (mutex u otro mecanismo)	10
Análisis de Concurrencia	Identificación de al menos dos condiciones de carrera en el servidor	5
	Explicación de cómo fueron evitadas (uso de sincronización)	5
Protocolo	Protocolo claro, consistente y funcional	10